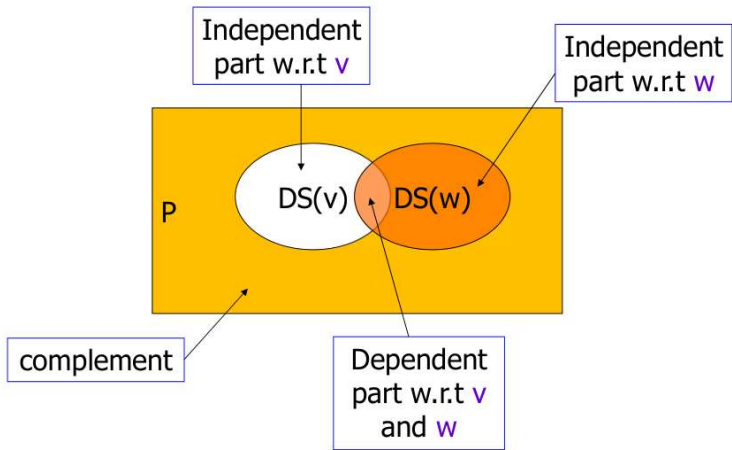


第六节&第七节

2025年6月9日 15:15

1. 回归测试：

- (1) 保证软件在演化和维护时，未改变代码的功能不受影响
 - (2) 在对之前测试过的程序修改后，仅根据修改重新测试，同时保证和完全重测相同的测试覆盖率
- 背景：软件改变作为软件开发乃至使用过程中的常规活动（甚至是日常活动），促进了软件升级换代的需要。
- 地位：是软件测试体系（理论、方法、技术手段）的必要组成



与一般测试的区别：

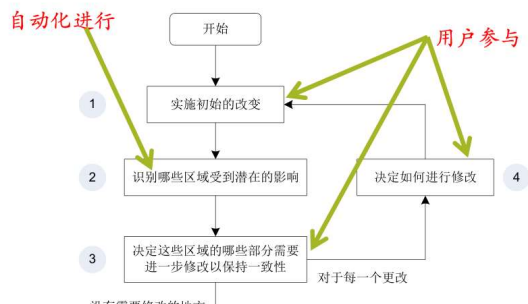
测试计划	面临修改过的SRS、修改过的程序和一个需要更新的旧的测试计划
测试范围	检测被修改的相关部分（而不是整个程序）有无缺陷
资源分配	所需时间、资源根据开发具体情况进行
开发信息	在不同时间、地点上进行，及时记录开发信息以保障正确进行
完成时间	所需时间更少，因为回归测试只测一部分，且依据脚本自动化
执行频率	在一个系统的生命周期内多次进行，系统被修改就开始进行

3. 软件被修改，与软件相关的所有东西都可能被修改，可能发生在任何时候，例如软件开发、维护阶段。
- 波及效应分析是为了发现所有（包括所有潜在的）受影响的部分，以保证软件发生改变后仍然保持一致性和完整性

波及效应分析的类型： { 需求的, 设计的, 代码的, 测试用例的 } , 各阶段间也要进行波及效应分析

过程：

- (1) 实施初始改变
- (2) 识别潜在的受影响的组件
- (3) 决定这些受到潜在影响的组件中哪些需要改变
- (4) 决定如何进行这些改变，对于每一个改变都要从第1步开始重复。如果没有要进行改变的，则结束



2. 回归测试流程：

- (1) 提出修改需求
- (2) 修改软件工件（软件制品：文档、代码、模型）
- (3) 选择和生成测试用例
- (4) 执行测试
- (5) 识别失败结果
- (6) 确认错误
- (7) 排除错误

策略： { 全部重新测试, 有选择的重新测试 }

全部重新测试：

优点：不耗精力去选择测试用例，适用于测试用例数目不太多或系统大部分被改变的情形

缺点：不适用于测试用例数目很多，而系统改动很微小

有选择的重新测试：

优点：适用于测试用例数目很多，而系统改动很微小

缺点：耗精力去选择测试用例，不适用于测试用例数目不太多或系统大部分被改变的情形。

选择常用算法：与某个特定模块相关的测试用例，保持固定特征（未被波及）不变

可追溯性原理：从需求到代码，然后再到测试用例，都需可追溯
所有黑/白盒的测试用例都要可追溯；因为软件工件间的依赖性，所以需要使用波及效应分析来确认那些被影响到的部分

4. 回归测试测试用例的生成：

- (1) 为了测试新改变代码，生成新的测试用例T1
- (2) 对原有的测试用例组进行有效性重确认T->T2
- (3) 执行测试用例组
- (4) 比较测试用例的执行结果与预期结果
- (5) 回溯失败，查明导致失败的模块或修改

建议：

- (1) 使用工具有好处。例如测试执行工具、测试结果比较器、配置工具、测试管理工具
- (2) 回归测试应考虑两种依赖性：模块依赖、修改依赖

5. 软件性能（非功能特性）：软件在运行时表现出来的时间和空间效率与用户需求之间的吻合程度。

软件性能是软件运行时完成某一功能的响应性特征，以及增加软件功能时保持响应性的能力，即可伸缩性。

用户需求只包含功能需求和非功能性需求。

用户视角：性能是响应时间（主客观、心理因素）和资源消耗。

管理员视角：普通用户感受到的软件性能+如何用管理功能进行性能调优

开发人员视角：管理员视角+开发过程中尽可能开发出高性能的软件
性能测试：通过自动化的测试工具模拟多种正常、峰值以及异常负载条件来对系统的各项性能指标进行测试

目的： { 验证软件系统是否能够达到用户提出的性能指标, 发现软件系统重存在的性能瓶颈, 优化软件, 优化系统 }

常见类型： { 基准测试, 负载测试, 压力测试, 容量测试 } 工具： { 负载压力测试工具, 功能回归测试工具, 测试管理工具 }



方法：{ 字符串匹配或交叉引用
程序切片

字符串匹配：不考虑程序执行，需要人类智慧进行REA操作。（再工程工具）

程序切片方法：确认直接和间接的波及，可以设计测试工具自动产生程序切片。（前向切片）

波及效应的分类：（直接波及是波及效应分析过程中要进行进一步改变的第一候选集）

{ 直接波及：被初始改变影响的，对于改变点有直接的数据或控制依赖
诱发波及：是由直接波及和其他诱发波及引起的（传导效应）

6. 性能测试的步骤：{ 熟悉应用
测试需求
测试准备
测试执行
测试结果分析

熟悉应用：整个性能测试过程最关键的一步，需要了解应用系统架构和应用系统的功能逻辑

测试需求：原始需求和目标需求都要声明

测试准备：{ 测试客户端机器准备
测试数据准备
测试脚本准备
测试用例准备

（1）测试客户端机器准备要充足，要和服务器保持通畅，如果瓶颈在客户端或者网络，则无法评估服务端。

（2）测试数据准备：如果被测功能涉及数据库和高速缓存，通常需要预设很大的数据量才能凸显性能瓶颈。

上线应用数据从线上拷贝，未上线应用需构造。测试数据准备的脚本有时多于测试脚本本身。若无法构造大数据量，则测试高速缓存，按照数据量比例减少高速缓存。

（3）测试脚本准备：测试脚本的实现往往会花费较长的时间，因其涉及到应用实现细节，需要和开发人员不断交流才能完成。

（4）测试用例准备：在了解软件业务流程的基础上设计测试用例。原则是可以满足预期性能指标测试用例需求的，就没有必要设计更多的内容，因为用例越多，执行的成本越高；目标是一次尽可能包含多个测试要素。

测试用例必须是测试工具可以实现的，不同的测试用例（场景）测试不同的性能。要尽可能把性能测试用例设计地复杂，才有可能发现软件的性能瓶颈。

测试执行：需要监控测试客户端和服务端性能、应用情况

测试结果分析：与测试监控息息相关，在测试执行的过程中，用各种监控工具看到系统运行状态，并及时发现问题。常见问题包括：

{ 内存问题
有限资源竞争问题：一个共享资源在一个时间点只可被一个线程获得

常见类型：{ 基准测试
负载测试
压力测试
容量测试

工具：{ 负载压力测试工具
功能回归测试工具
测试管理工具

基准测试：在给系统施加较低压力时，查看系统的运行情况并记录相关数据作为基础参考。

负载测试：检测在各种工作负载下系统的性能，即当负载逐渐增加时，系统各项性能指标的变化情况

压力测试：检测一个系统的瓶颈或者不能接受的性能点。测试系统能够提供的最大级别服务

容量测试：检测系统在给定时间内能够持续处理的最大负载或工作量

容量：容量测试通过测试找出能够反映软件系统应用特征的某项指标的极限值，在这个极限状态下没有出现任何软件故障或还能保持主要功能正常运行。其中极限值就是容量，容量测试还将确定测试对象在给定时间内能够持续处理的最大负载或工作量。

7. 性能测试的指标

{ 响应时间：系统对请求做出响应的时间
系统响应时间：计算机对用户输入或请求做出反应的时间
应用响应时间：应用接到指令后的处理时间
吞吐量：系统在单位时间内处理请求的数量
并发用户数：系统可以同时承载的正常使用系统功能的用户数
资源利用率：一段时间内资源平均被占用的情况

响应时间=网络传输时间+应用服务器处理时间+数据库服务器处理时间

系统用户数：系统额定的用户数量

同时在线用户数：在一定的时间范围内，最大的同时在线用户数量

（平均）并发用户数：同时在与服务器进行交互操作的用户总数量

吞吐量：{ 业务角度：RPS、页面数每秒、人数每天、TPS，反应服务器
网络角度：字节数每秒

承受的压力，不同方式表达的吞吐量可以说明不同层次的问题。

资源利用率：系统各种资源的使用情况，一般使用“资源实际使用/总的

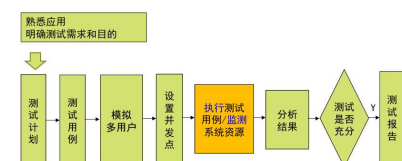
资源可用量”形成资源利用率：{ CPU
平均负载
内存
队列
IO
网络

内存 { 在使用字节
可使用字节数
页面错误每秒
页面每秒
网络 { 总字节每秒
包每秒
磁盘 { 磁盘时间
空闲时间
平均磁盘队列长度
当前磁盘队列长度
处理器 { 处理器时间
用户时间
处理器队列长度

10. 压力测试的主要特点：{ 超出正常负载
极限条件
故障恢复
性能降级

主要目标：{ 识别系统的瓶颈和弱点
确定系统的最大容量
验证系统在极端条件下的稳定性
评估系统在过载后的恢复能力
确保关键功能在压力下仍能工作

压力测试是确保软件系统可靠性和韧性的可靠手段



8. 软件性能测试方法:

- 基准测试
- 负载测试
- 压力测试
- 容量测试
- 配置测试
- 并发测试
- 可靠性测试
- 失效恢复测试

基准测试：有基础标准，通过对比发现系统的不同点。可在制定的基础标准下通过基准测试建立一个性能标准，这样后续当系统的环境、参数发生变化之后，再进行一次相同标准下的测试，即可看出变化对性能的影响。可在较早的阶段发现性能问题。

负载测试：在被测系统上不断增加负载，直到性能极致。目的是找系统的负载极限，为系统调用提供数据

压力测试：目的是找出高负载下系统的问题

容量测试：确定测试对象在给定时间内能持续处理的最大负载或工作量

配置测试：通过被测系统软硬件环境的调整，了解各种环境对系统性能影响的程度，从而找到各项资源的最优分配原则

并发测试：通过模拟用户并发访问，测试多用户并发访问同一应用、模块或数据记录时是否存在死锁或者其他性能问题。并发用户数≠并发数

可靠性测试：给系统加载一定的业务压力，让应用系统持续运行一段时间，测试在这种条件下是否能够稳定运行

失效恢复测试：针对有备份和负载均衡的系统。用来检验如果系统局部发生故障，用户能否继续使用系统，以及如果这种情况发生，用户将受到多大程度的影响。